

QPROG Project

Dominating Set Using Grover's Algorithm

Mateus de Oliveira Oliveira
Department of Computer and Systems Sciences
Stockholm University

1 Dominating Set

Let $G = (V, E)$ be a graph with vertex set V and edge set E . A *dominating set* of G is a subset $S \subseteq V$ with the property that for each vertex $v \in V$, either v belongs to S , or there is some $u \in S$ such that $\{u, v\} \in E$. That is to say, each vertex of G is either in S or is connected by some vertex in S .

Definition 1 (Dominating Set Problem). *In the Dominating Set problem, we are given a graph G and an integer k , and the goal is to determine whether G has a dominating set of size k .*

The Dominating Set problem has a large number of applications, ranging from sensor networks to logistics. In this project, you will be using Grover's algorithm to search for solutions for a given instance of the dominating set problem. A naive brute force algorithm can solve this problem with at most $\binom{n}{k} \leq n^k$ calls to a verifier for the dominating set problem. Enumerate all possible subsets of size k in G , and for each of the sets, verify whether it is a dominating set. Using Grover algorithm, we are aiming at an algorithm that solves the problem with roughly $n^{k/2}$ calls to the verifier algorithm.

This project is essentially split into two parts. First, given a graph G and an integer k , you should implement a quantum circuit that takes a list of k vertices as input, and returns 1 in the output bit if and only if the vertices in this list form a dominating set of G . In other words, this circuit is a verifier for the dominating set problem. In the second part of the project, you should combine this circuit with Grover's algorithm to search for an actual solution.

Instructions

- Unless stated otherwise, for the implementation of circuits implementing a Boolean function F , you are only allowed to use the gates **X**, **CNOT**, **CCNOT** and **Multicontrolled-NOT**. See Appendix A for an example of use of the **Multicontrolled-NOT** gate.
- Please feel free to implement additional auxiliary functions as long as the functions listed below are implemented.
- For each implemented function, please give evidence that the implementation is correct, by initializing each input register with some number of up to 4 bits, each auxiliary register with $|0\rangle$, and by measuring the output register, to verify if the value is as expected. **The tests should be described in a PDF report. If for some reason you were not able to run the simulation of a function (for example, if the number of qubits became too large), then specify this in your report.**
- Please try to reuse qubits from auxiliary registers as much as possible. For this, it is crucial that the auxiliary registers are equal to $|0\rangle$ both at the beginning and at the end of the computation of each function.
- The value of each question is expressed as a percentage of the total points for the project.

Glossary

1. `circuit` denotes the quantum circuit to be modified by the function.
2. Lower case letters `a,b,c,...` denote integers specifying single-qubit registers. The letters `x,y,z,...` denote bits.
3. Upper case letters `A,B,C,...` denote lists of integers, which intuitively specify qubit registers used to store the input and output of functions. The letters `X,Y,Z,...` denote numbers represented in binary. `len(A)` denotes the length of the list A . `A[0]` is the least significant bit of the register, while `A[len(A)-1]` is the most significant bit. `bin(A)` denotes the binary string stored in the register A . `number(A)` denotes the number represented by `bin(A)`.
4. `AUX` is a list of integers which is intuitively used to store the partial results of the computation. Unless stated otherwise, it is assumed that `aux` is initialized with the state $|0\rangle$ before the function is applied, and that this register is reset to $|0\rangle$ once the function has been applied.

1.1 Defining a Graph Class (12.5%)

Define a class `Graph` to store graphs. The class has two attributes:

- `n`
- `adj_list`

The first attribute, `n`, is the number of vertices of the graph. The second attribute, `adj_list`, is the adjacency list of the graph (that is, a list containing `n` sublists). Vertices are numbered from 0 to `n-1`. For each $v \in \{0, \dots, n-1\}$, the v -th sublist is the set of neighbors of vertex v . For example, if $G = (V, E)$ with $V = \{0, 1, 2, 3, 4\}$ and $E = \{\{0, 1\}, \{0, 3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}\}$ then we G corresponds to the graph instance where `n=5` and `adj_list = [[1,3], [0,2,3], [1,3], [0,1,2], []]`. Note that the last vertex, 4, is not connected to any other vertex.

The class should contain the following methods:

- `print()`: prints the graph. Print it in the way you think is best.
- `set_number_vertices(n)`: sets the number of vertices of the graph to n
- `add_edge(u,v)`: adds edge $\{u, v\}$. Note that u should be added to the sub-list corresponding to v and v should be added to the sub-list corresponding to u .
- `read_from_file`: reads the graph from a file. The file format is as follow. The first line is a single number specifying the size of the graph, that is n . Each of the subsequent lines has two numbers between 0 and $n-1$ specifying an edge of the graph. The two numbers should be separated by a single whitespace. Please use the extension `".txt"` for the graph files, even if you are working on linux or mac.

1.2 Adjacency Circuit (12.5%)

Implement a function `Adj(G,circuit,A,B,b)` that sets qubit b to 1 if and only if $\{\text{number}(A), \text{number}(B)\}$ is an edge of G . Assume that $G.n$ is a power of 2 and that A and B are lists with $\log_2 G.n$ bits. **Hint 1:** this circuit does not need auxiliary bits. The whole circuit is a sequence of multicontrolled gates, two per edge, where the control qubits are those in the lists A and B and where the target qubit is qubit b . **Hint 2:** There are two multicontrolled gates per edge because the graph is undirected.

1.3 Dominated Vertex (12.5%)

Implement a function `Dominated(G,circuit,A_1,...,A_k,B,AUX,b)` that sets qubit b to 1 if and only if $\text{number}(B)=\text{number}(A_i)$ or $\{\text{number}(B), \text{number}(A_i)\}$ is an edge of G , for some $i \in \{1, \dots, k\}$. Intuitively, this function tests whether vertex $\text{number}(B)$ has been dominated by some vertex in the set $\{\text{number}(A_1), \dots, \text{number}(A_k)\}$.

- **Hint 1:** You need to test whether $\{\text{number}(B), \text{number}(A_1)\}$ is an edge, OR whether $\{\text{number}(B), \text{number}(A_2)\}$ is an edge, ..., OR $\{\text{number}(B), \text{number}(A_k)\}$ is an edge. Additionally, test whether $\text{number}(B)=\text{number}(A_1)$, OR $\text{number}(B)=\text{number}(A_2)$, ..., OR $\text{number}(B)=\text{number}(A_k)$.
- **Hint 2:** You should compute this sequence of OR operations in a clever way. This means that you should use the same auxiliary register for each OR test. For this, the auxiliary register should start at $|0\rangle$ and be reset to $|0\rangle$ after each OR.
- **Hint 3:** Use the output qbit (that is, b) to store the result of the OR test. This qubit starts at $|0\rangle$. If it is set to $|1\rangle$ by some OR in the sequence, it will keep being $|1\rangle$ until the end.

1.4 All Dominated (12.5%)

Using the previous function, implement a function `AllDominated(G,circuit,A_1,...,A_k,AUX,b)` that sets qubit b to 1 if and only if every vertex of the graph is dominated by some vertex in the set $\{\text{number}(A_1), \dots, \text{number}(A_k)\}$. Intuitively, the resulting circuit is a verifier for the dominating set problem (with k fixed).

- **Hint 1:** The circuit implements a sequence of AND operations.
- **Hint 2:** Again, reset the auxiliary register after each AND.

1.5 Grover with One Solution (12.5%)

Now that you have the circuit for the verifier, use Grover's algorithm to determine whether there is some subset S of size k that dominates all vertices of the graph. Note that in total, your input register has $k \log_2 n$ qubits, where n is the size of the graph. Therefore, the search

space has size $2^{k \log_2 n} = n^k$. This means that if there is a single solution, one should iterate the diffusion and oracle operators about $\pi/4\sqrt{n^k}$ times.

1.6 Experimental Evaluation: One Solution (12.5%)

Design experiments to evaluate the performance of your implementation. For this, create instances of graphs with 4 nodes, 8 nodes and 16 nodes, and with dominating sets of size 2 and 4. For this, you should come up with instance with a single dominating set of the required size. Use any method you want to create these instances (ex: trial and error).

1.7 Grover with Multiple Solutions (12.5%)

If there are multiple solutions, we need to estimate the number of iterations. For the purposes of this project, it is enough that you perform a very simple estimation as follows:

- Start by applying Grover's algorithm with a single iterations.
- If the state given after measurement is a solution, we are done.
- If not, call Grover's algorithm again, but now with the double of the number of iterations.
- Repeat this until you reach $\sqrt{n^k}$ iterations.

For simplicity, if the algorithm did not find a solution in any steps above, assume that there is no solution.

1.8 Experimental Evaluation: Multiple Solutions (12.5%)

Again, using small graphs, evaluate the performance of Grover's algorithm in the multiple solutions case.

A Multicontrolled NOT Gate

To be able to use multicontrolled NOT gates you need to import the `MCXGate` module. This gate applies the NOT gate to the target qubit if and only if all control qubits are set to $|1\rangle$. An example is provided below where the control qubits are 2, 3 and 4 and the target qubit is 7.

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import MCXGate
# Create a quantum circuit with at least 8 qubits
qc = QuantumCircuit(8)
# Define the number of control qubits
num_controls = 3
# Create an MCX gate
mcx_gate = MCXGate(num_controls)
# Apply the MCX gate (control qubits 2, 3, 4 and target qubit 7)
qc.append(mcx_gate, qargs=[2, 3, 4, 7])
# Draw the circuit
print(qc)
```